

Visualization Draft

Phoebe Dupa, Rebecca Martinez, and Shucan Zhao

```
# general use  
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
v dplyr      1.1.4      v readr      2.1.6  
v forcats    1.0.1      v stringr    1.6.0  
v ggplot2     4.0.3      v tibble     3.3.0  
v lubridate  1.9.4      v tidyr      1.3.2  
v purrr       1.2.1  
-- Conflicts ----- tidyverse_conflicts() --  
x dplyr::filter() masks stats::filter()  
x dplyr::lag()     masks stats::lag()  
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(janitor)
```

Attaching package: 'janitor'

The following objects are masked from 'package:stats':

chisq.test, fisher.test

```
# comparing groups  
library(flextable)
```

Attaching package: 'flextable'

The following object is masked from 'package:purrr':

compose

```
library(ggribes)

# file organization
library(here)
```

here() starts at C:/github/ENVS-193DD-Final_Project

```
# weather data from NOAA weather station
weather <- read_csv(here("data", "NOAA-weather-data.csv"))
```

Rows: 1912 Columns: 9

-- Column specification -----

Delimiter: ","

chr (3): STATION, NAME, DATE

dbl (6): LATITUDE, LONGITUDE, ELEVATION, PRCP, TMAX, TMIN

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# water quality survey metadata
metadata <- read_csv(here("data", "NCOS_YSI_Water_Quality_Monitoring_0.csv"))
```

Rows: 687 Columns: 17

-- Column specification -----

Delimiter: ","

chr (12): GlobalID, Monitoring Date, Monitor Name, Weather, Site Name, Othe...

dbl (4): ObjectID, Water Surface Elevation, x, y

time (1): Time of WQ Measurement

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# water quality parameter measurements
water_parameters <- read_csv(here("data", "YSI_Data_Begin_1.csv"))
```

Rows: 1488 Columns: 13

-- Column specification -----

Delimiter: ","

chr (6): GlobalID, ParentGlobalID, CreationDate, Creator, EditDate, Editor

dbl (7): ObjectID, Depth Code, DO (mg/L), DO (%sat), Conductivity (uS/cm), S...

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
weather_clean <- weather |>
# cleaning column names
clean_names() |>
# making sure date is in POSIXct format
mutate(date = mdy(date)) |>
# selecting columns of interest
select(date, prcp, tmax, tmin) |>
# extracting month and year from date column
mutate(month = month(date),
       year = year(date)) |>
# assigning water year
mutate(wy = case_when(
  # when month is October or later, assign value of year + 1
  month >= 10 ~ year + 1,
  # when month is September or earlier, assign value of existing year
  .default = year))
```

```
# creating new object from metadata
metadata_clean <- metadata |>
# cleaning column names
clean_names() |>
# excluding unnecessary columns
select(!c(object_id, creator, editor)) |>
# making sure monitoring_date is in POSIXct format
mutate(monitored_date = mdy_hms(monitored_date)) |>
# renaming monitoring datetime column to reflect values
rename(monitored_datetime = monitored_date) |>
# creating new columns for month and year of observations
mutate(month = month(monitored_datetime),
       year = year(monitored_datetime)) |>
# assigning water year
mutate(wy = case_when(
  # when month is October or later, assign value of year + 1
```

```

    month >= 10 ~ year + 1,
    # when month is September or earlier, assign value of existing year
    .default = year)) |>
# create new column of full site name
mutate(site_full = case_when(
  site_name == "east_channel" ~ "East Channel",
  site_name == "phelps_bridge" ~ "Phelps Bridge",
  site_name == "venoco_bridge" ~ "Venoco Bridge",
  site_name == "other" ~ "Other"
))

```

```

# creating new object from water_parameters
water_parameters_clean <- water_parameters |>
# cleaning object names
clean_names() |>
# excluding unnecessary columns
select(!c(creator, editor)) |>
# renaming columns
rename(elevation_code = depth_code) |>
# replacing incorrect elevation code from 2021-07-16
mutate(elevation_code = case_when(
  elevation_code == 10 ~ 1,
  .default = elevation_code
)) |>
# set factors
mutate(elevation_code = as_factor(elevation_code),
       elevation_code = fct_relevel(elevation_code,
                                     "0", "1", "2", "3", "4", "5", "6"))

```

```

# join metadata and water parameters
water_quality <- full_join(
  # left
  x = metadata_clean,
  # right
  y = water_parameters_clean,
  # shared column
  by = c("global_id" = "parent_global_id")) |>
# stop and look at colnames(water_quality) before continuing
# selecting relevant columns
select(
  # information about observation
  global_id, monitoring_datetime, time_of_wq_measurement,

```

```

    monitor_name, weather, site_name, other_site_name, site_full, x, y,

    # water information
    water_surface_elevation, flow, notes,

    # measurement information
    elevation_code, do_mg_l, do_percent_sat, conductivity_u_s_cm,
    salinity_ppt, temperature_c) |>
# extract date from monitoring datetime
mutate(date = date(monitored_datetime)) |>
# move date column to appear to the right of monitored_datetime
relocate(date, .after = monitored_datetime) |>
# exclude monitored_datetime
select(!monitored_datetime)

```

```

# join water quality and weather
water_quality_weather <- full_join(
  # left
  x = water_quality,
  # right
  y = weather_clean,
  # shared column
  by = "date"
)

# left join (exclude rows that don't match in weather)
water_quality_weather <- left_join(
  # left
  x = water_quality,
  # right
  y = weather_clean,
  # shared column
  by = "date"
)

```

```

do_filtered <- water_quality %>%
  # filter out "other" sites
  filter(site_full != "Other") %>%
  # filter out missing elevation code values
  filter(!is.na(elevation_code)) %>%
  #filter out missing DO values
  filter(!is.na(do_mg_l)) %>%

```

```
# reorder site levels from lowest to highest mean salinity
mutate(site_full = fct_relevel(
  site_full,
  "Phelps Bridge", "East Channel", "Venoco Bridge"))
```

```
# summarize dissolved oxygen across sites
do_site_summary <- do_filtered %>%
  # group by site
  group_by(site_full) %>%
  # calculate summary statistics for dissolved oxygen
  summarize(
    mean_do = mean(do_mg_l, na.rm = TRUE),
    median_do = median(do_mg_l, na.rm = TRUE),
    sd_do = sd(do_mg_l, na.rm = TRUE),
    var_do = var(do_mg_l, na.rm = TRUE),
    n = n(),
    .groups = "drop")

# table
do_site_summary
```

```
# A tibble: 3 x 6
  site_full      mean_do median_do      sd_do var_do      n
  <fct>          <dbl>    <dbl>    <dbl>  <dbl> <int>
1 Phelps Bridge    5.62      5.40      3.32 1.10e 1    382
2 East Channel 304885.      8.10 5521576. 3.05e13    328
3 Venoco Bridge 1426725.      6.98 36189088. 1.31e15    771
```

Visualizing Figures

More Wrangling (if needed)

```
do_filtered <- do_filtered |>
  mutate(log_do_mg_l = log(do_mg_l) + 1) # makes sure output is numeric
```

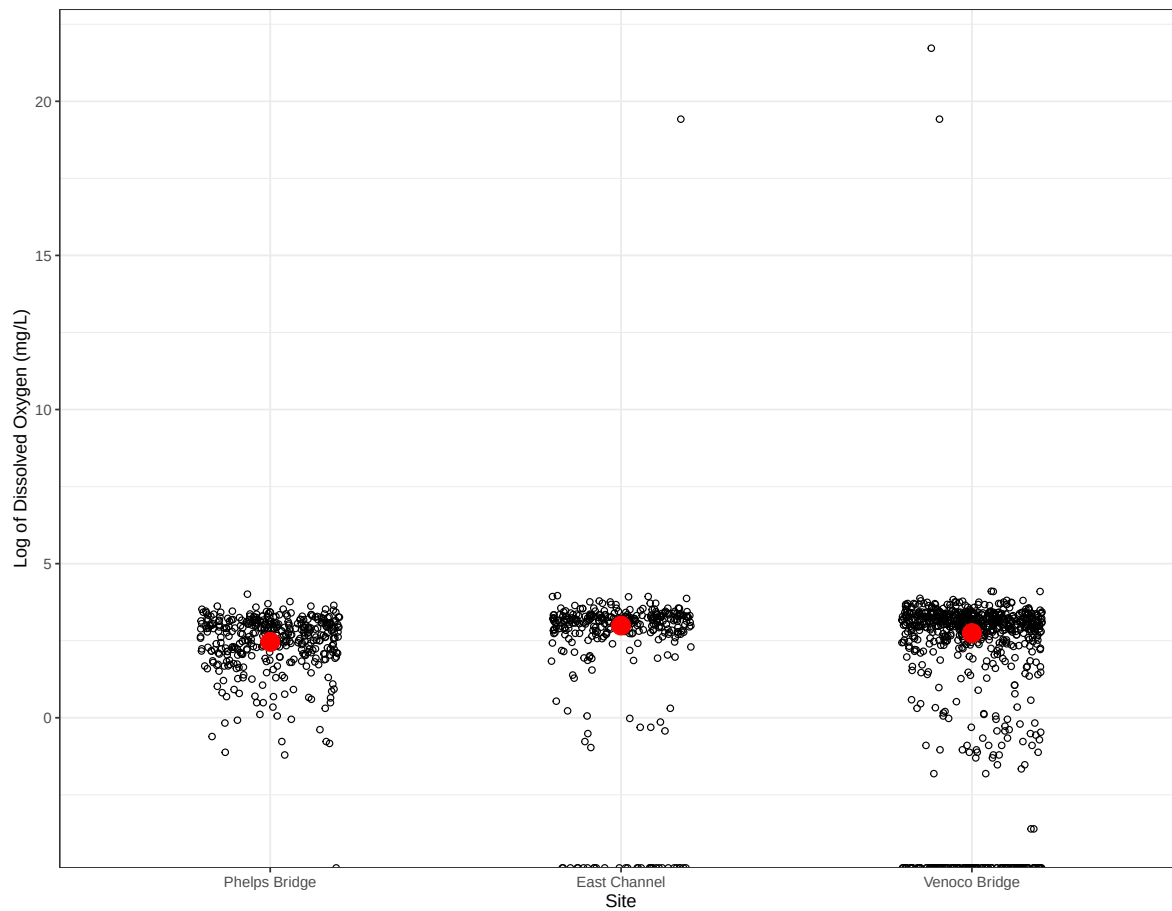
```

# base layer: ggplot
ggplot(data = do_filtered,
       # x-axis: site
       # y-axis: salinity
       mapping = aes(x = site_full,
                     y = log_do_mg_l)) +
# jitter to represent individual salinity measurements
geom_jitter(height = 0,
            width = 0.2,
            shape = 21) +
# quick calculation of mean for visualization
stat_summary(fun = mean,
            geom = "point",
            color = "red",
            size = 5) +
# relabelling x- and y-axes, adding title
labs(x = "Site",
     y = "Log of Dissolved Oxygen (mg/L)",
     title = "Sites differ in amount of dissolved oxygen across all
↵ timepoints") +
# different theme from default
theme_bw()

```

Warning: Removed 170 rows containing non-finite outside the scale range (``stat_summary()``).

Sites differ in amount of dissolved oxygen across all timepoints



2

```
# using do_filtered data frame
ggplot(data = do_filtered,
  # x-axis: date
  # y-axis: salinity
  # color by elevation code
  mapping = aes(x = temperature_c,
    y = log_do_mg_l,
    color = elevation_code)) +
  # points to represent measurements
  geom_point() +
  # separate panels for each site
```

```

facet_wrap(~ site_full) +
# custom colors
scale_color_manual(
  values = c("0" = "darkgreen",
            "1" = "goldenrod",
            "2" = "turquoise")
) +
# relabelling x-and y-axes, legend
labs(x = "Temperature (Celsius)",
     y = "Log of Dissolved Oxygen (mg/L)",
     color = "Elevation code") +
# different theme
theme_bw()

```

